

Natural Language Processing

Tel Aviv University

Assignment 3: Advanced Topics

Due Date: *July 14th, 2026*

Lecturer: Dr. Mor Pipek, TA: Idan Tarshish

Instructions

Important Note: This assignment involves interfacing with LLMs via a free-tier API, which entails specific rate and usage limits. To ensure a smooth workflow, we recommend the following:

1. Utilize separate API accounts for Question 1 and Question 2 to maximize your available quota.
2. Begin the implementation well in advance of the deadline. This will allow for sufficient buffer time should you encounter rate-limiting delays or service throttling.

Submission Instructions

This assignment includes both theoretical and coding questions. Your submission should be one zip file that includes:

1. A single PDF file in digital format (we recommend using Overleaf, but you may use any digital editor you prefer). The PDF should contain:
 - (a) Answers to Q1: 2.c, 3.b, 3.c, 4.d, 4.e, 5.c, 6.c.
 - (b) Answer to the Analysis question Q2
 - (c) Answers to all questions of Q3 as written in 3.3.
 - (d) AI disclosure (Q4)
2. The directory whodunnit, including all its files, with the **completed** agents.py file.
3. The **completed** notebook prompting_architectures.ipynb.
4. The **completed** notebook textual_search.ipynb.

Submit your solution via Moodle. Your submission should be a single zip file named <id1>_<id2>_<id3>.zip, where id1 is the ID of the first student. The zip file should include all of the above components.

Groq API Setup

Groq is a high-performance inference platform that provides extremely low-latency access to modern Large Language Models. In this assignment, you'll use the Groq API to access and work with some of the latest LLMs available.

Account Setup and API Key

1. **Registration:** Navigate to console.groq.com and sign up using your academic email or Google account. **No payment details are required** to access the initial free tier. Furthermore, the requirements for this assignment are designed to stay within the constraints of the free usage limits; you will **not** be expected to upgrade to a paid plan.
2. **Key Generation:** Go to the **API Keys** section in the sidebar (or visit console.groq.com/keys).
3. **Create Key:** Click "Create API Key", give it a descriptive name (e.g., `NLP_Assignment_q1`), and click Submit.
4. **Secure Storage: Copy the key immediately.** It will only be displayed once. Store it securely in your local environment or within the `groq_utils.py` file as instructed.

Usage

For both questions, the API connection logic has been pre-implemented. To begin, ensure that the `groq` package is installed (`pip install groq`) and that you are using Python 3.10 or higher. You must then insert your API key into the designated field within the provided files.

The `groq_utils.py` script abstracts the majority of the boilerplate code. For conceptual clarity, a standard chat completion request using the Groq client is structured as follows:

```
from groq import Groq

client = Groq(api_key="YOUR_API_KEY")

completion = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Is the sky blue?"}
    ],
    temperature=0.7,
)

print(completion.choices[0].message.content)
```

While all credentials will be ignored during the grading process, you are encouraged to remove your API key from the source files prior to submission for security purposes.

Rate Limits and Best Practices

The free tier is generous but strictly enforced. As of 2026, the `llama-3.1-8b-instant` model typically allows:

- Requests Per Minute (RPM): 30

- Requests Per Day (RPD): 14,400
- Tokens Per Minute (TPM): 30,000

If you receive a 429: Too Many Requests error, implement a small delay (`time.sleep`) between your API calls or reduce the frequency of your test runs. You can read more details about the rate limits [here](#).

Pro-Tip: Use the [Groq Playground](#) to test your prompts manually before implementing them in your code to save on API quota.

1 Prompting for Reasoning (35 pt)

In this section, we will explore the ability of Large Language Models (LLMs) to reason over *complex, multi-step questions*. To do this, we will use the **Groq** platform. Please make sure you begin this exercise **only after completing the Groq setup instructions** provided at the beginning of the assignment.

We will use the **StrategyQA** dataset, a benchmark dataset designed to evaluate multi-hop and common-sense reasoning in question answering, as introduced by Geva et al. 2021. Each example in StrategyQA contains a yes/no question that typically requires combining multiple pieces of knowledge or reasoning steps to answer correctly.

For example, consider the question: “*Do the anchors on Rede Globo speak Chinese?*” Answering this question requires reasoning over several facts, such as knowing that *Rede Globo* is a Brazilian television network and that the primary spoken language in Brazil is Portuguese rather than Chinese. Only by combining these pieces of knowledge can one arrive at the correct answer.

The attached file `strategyqa_testset_50.json` contains 50 selected examples from the StrategyQA dataset. Throughout the following sections, whenever you are asked to evaluate model performance, you should report the **accuracy computed over these 50 examples**.

For each of the following questions, use **llama-3.1-8b-instant** as your model, unless you are explicitly instructed to use GPT-2.

Load the attached notebook into Google Colab and complete the following tasks:

1. Setup

- Upload both attached JSON files to your Colab environment.
- Insert your Groq API key in the designated location and verify that the connection works correctly.
- Carefully review and understand the functions `query_llama` and `query_gpt`. These functions serve as the core interface for querying the models and will be used throughout the assignment.

2. Zero-shot

- Implement the function `zeroshot`. Use LLaMA with `temperature = 0` (i.e., greedy decoding), and do not modify the system prompt in this subsection.
- Implement the function `evaluate`. This function should return 1 if the model’s prediction matches the correct answer and 0 otherwise. Note that a naive string comparison (e.g., `pred`

`== answer`) may be insufficient due to formatting or phrasing variations in model outputs. Since `evaluate` will be reused with additional prompting methods later in the assignment, design it to be robust and general.

- (c) Implement the function `zeroshot_gpt` using the preloaded GPT-2 model.
- (d) **In your written solution**, report the zero-shot accuracies for both LLaMA and GPT-2. Then, describe 3 differences between these **models** that could explain the performance gap.

3. Self-Consistency

Language models are often inconsistent in their responses: they may produce different answers to the same question (or to slightly different phrasings of it), and they may sometimes assign relatively high probability to incorrect answers. To mitigate this issue, we will use a *self-consistency* strategy that aggregates multiple sampled responses.

- (a) Implement the `self_consistency` function. For each query, sample the model $n = 5$ independent times using a `temperature` of 1.0. The final prediction should be determined via a majority vote across these samples.
 - Since semantically identical answers may manifest in different formats (e.g., "yes" vs. "True"), you are encouraged to alter the system prompt, or to use post-processing to aggregate equivalent responses before voting.
 - You may implement any logically sound tie-breaking strategy.
- (b) Report the final accuracy achieved with `temperature = 1` in your written PDF solution.
- (c) Repeat the process using `temperature` settings of 0 and 0.5. Analyze the internal voting distribution for each setting. What insights does this provide regarding the relationship between the `temperature` parameter, model confidence, and output diversity? You may add codeblocks and relevant console outputs to support your analysis.

4. In-Context Learning (ICL)

Another powerful technique for improving model performance is *in-context learning (ICL)*, where we provide the model with several solved examples (demonstrations) inside the prompt. The model can then infer the desired reasoning pattern and apply it to new questions.

- (a) You are given a dictionary `demos_dict` containing 50 demonstration examples (loaded from the attached json `strategyqa_demo_questions_50.json`). Inspect a few examples to understand their structure and content.
- (b) Implement the function `build_demo_prompt`, which converts a single demonstration example into a formatted prompt segment. You may use **only** the demonstration's `question`, `answer` (for now).
- (c) Implement the function `icl_pred`, which constructs a full prompt by concatenating n formatted demonstrations (using `build_demo_prompt`) followed by the new question. Query the model with this prompt to obtain the prediction. Choose an appropriate temperature for ICL.
- (d) Evaluate the ICL method with $n = 10$ on the dataset and report the resulting accuracy in your written solution.
- (e) In your written solution, evaluate how the quantity and the sequential ordering of examples within the prompt influence model performance. Additionally, discuss the impact of the content of these examples, specifically addressing the relevance of their similarity to the target queries. You may add codeblocks and experimental results to support your analysis.

5. Chain-of-Thought

Another technique for improving reasoning in language models is *chain-of-thought (CoT) prompting*. Instead of asking the model to produce only a final answer, we encourage it to generate intermediate reasoning steps that lead to the conclusion. This often improves performance on multi-step reasoning tasks by making the model's internal reasoning more explicit.

- (a) Choose an appropriate **system prompt** that encourages the model to reason step by step before producing its final answer. Design your own prompt, but it should explicitly instruct the model to explain its reasoning and to clearly state the final yes/no (or True/False) answer at the end.
- (b) Implement the function `cot_pred`, which queries the model using your chosen chain-of-thought prompt and returns the predicted answer. When calling the model, allow a sufficiently large maximum token budget so that it has room to generate intermediate reasoning steps. You should also choose an appropriate temperature. Because the model's output will now include intermediate reasoning in addition to the final answer, implement a post-processing step that reliably extracts the final yes/no prediction in a consistent format. Your extraction method should be robust to minor variations in phrasing and formatting.
- (c) Evaluate your CoT performance on the dataset and report the resulting accuracy in your written solution.

6. Chain-of-Thought + In-Context Learning

Finally, we aim to integrate both techniques to optimize the model's predictive accuracy.

- (a) Implement the function `build_cot_demo_prompt`. In this version, you should leverage also the demonstration's `facts` and `decomposition` fields, besides the already used `question` and `answer`. Your objective is to design a clear, consistent structure that demonstrate the model how to reason through the problem before presenting a final answer. Refer to Figure ?? for a reference template.
- (b) Define appropriate `system_prompt` tailored for this hybrid task.
- (c) Implement the function `icl_cot_pred`, incorporating the insights gained from previous subsections.
- (d) Evaluate the performance of your combined CoT+ICL approach on the dataset. Report the final accuracy in your written solution.
- (e) Did the combination of ICL and CoT improve your results compared to using each technique in isolation? Provide a detailed explanation of how the synergy between these methods affected performance and why you believe these results occurred. Write your answers in your written solution,

2 WhoDunnIt: Multi-Agent Systems (35 pt)

In this section, you will experiment with autonomous agents in a simplified version of the *WhoDunnIt* game, as introduced by Barbi, Yoran, and Geva [2025](#).

Game Rules

The game consists of an Environment (Env) that generates a list of suspects, each defined by specific traits and attributes (refer to `attributes_small.json`). One suspect is designated as the culprit. The objective is for the agents to identify and accuse the correct individual.

We utilize an **Asymmetric Environment** consisting of two agents with distinct knowledge bases and actions:

- **The Accuser:** Knows the description of the culprit but has no information regarding the list of suspects. This agent must ask questions to identify the culprit. It has 3 optional actions:
 - request-specific - ask a question about a specific Suspect
 - request-broad - ask which suspects have a given trait/attribute
 - accuse - accuse a suspect. if the culprit is accused, the game ends and the agents won.
- **The Intel:** Has access to all suspect descriptions but is unaware of which one is the culprit. This agent must accurately answer the Accuser's inquiries. It has 2 optional actions:
 - respond - answer a specific question from the accuser
 - request-broad - answer a broad question from the accuser.

The game concludes when the Accuser successfully identifies the culprit or when the maximum number of turns is reached. Agents communicate via a shared channel (`Env.comm_channel`).

Setup and Configuration

To begin, extract the files from `whodunnit.zip` into your development environment. A Python version of 3.10 or higher is recommended. For this assignment, we will use Llama models via the **Groq API**. Please make sure you begin this exercise **only after completing the Groq setup instructions** provided at the beginning of the assignment.

1. Install the required library by running `pip install groq`.
2. Generate an API key at the [Groq Console](#) and insert it into the designated variable within `groq_utils.py`.
3. Execute `groq_utils.py` to verify that your connection is properly configured.

Models

For this assignment, we will use the `llama-3.3-70b-versatile` model. Due to the restrictive rate limits of this model, we recommend utilizing the smaller `llama-3.1-8b-instant` model for debugging and testing purposes. This can be achieved by setting the flag `use_small_model=True` within the `query_llama` function. Please be aware, however, that the smaller model lacks the capacity to successfully solve the game.

Note: All final results reported in your submission and used for your analysis must be generated using the larger `llama-3.3-70b-versatile` model.

2.1 Naive Agent Implementation

The provided baseline for the Accuser and Intel agents forces them to respond using only their private knowledge and the shared communication history, without an explicit reasoning step. By analyzing this baseline, we aim to uncover the inherent challenges and emergent behaviors within the system.

Instructions:

- Examine the `take_turn_naive` function within the `IntelAgent` and `AccuserAgent` classes in `agents.py`. Ensure you understand the `query_llama` function parameters, as well as the specific user and system prompts used for this baseline.
- Carefully analyze the logic in `Env.handle_agent_outputs` to understand how the environment parses agent responses and manages the communication channel.
- **Important:** Do not modify any code files during this section.
- Execute the game using the command `python run_game.py` with default parameters; the full execution history will be logged in `game_log.txt`.
- **Note:** Do not use the `--include-summarizer` or `--allow-internal-thought` flags at this stage.

Analysis: Examine the `game_log` file to debug and evaluate agent behavior. Perform several runs and observe emerging patterns or failures. **In your written solution, address the following:**

- What specific behavioral problems did you identify in the initial run of the naive agents? Provide at least two concrete examples from your `game_log`.
- Do the agents dynamically utilize the full spectrum of their available action space, or do they prematurely converge on a single, repetitive action type? Analyze how this behavioral pattern affects their overall progress.
- Does the Accuser make a blind accusation prematurely, or does it get stuck in an endless loop until the `max-turn-count` is reached? Why do you think it struggles to finalize the decision?

2.2 Context Management via Summarization

As the game progresses, the growing context window may degrade agent performance. To address this, we introduce a third agent: the **SummarizerAgent**. While summarization is often handled internally by the primary agents, we have decoupled this functionality into a specialized agent to simplify the architecture.

The Summarizer has special access to the communication channel; rather than contributing new dialogue, it has the authority to edit the existing history. Its goal is to maintain a concise channel while preserving all critical details required by the Accuser and Intel agents. The Summarizer acts third in the rotation, executing its turn immediately after the Intel agent.

Instructions:

- Implement the `take_turn` function within the `SummarizerAgent` class.
- Use the `query_llama` function to process the history.
- **Caution:** Since the Summarizer can overwrite previous turns, design its prompts to be highly sensitive to specific evidentiary details to avoid losing crucial information.
- Enable the agent by including the `--include-summarizer` flag during execution.

Analysis: Examine the logs of the games played with the `SummarizerAgent` enabled. **In your written solution, address the following:**

- Did the summarizer inadvertently remove any critical clues that led to the Accuser failing to identify the culprit? Provide a specific example from the logs.
- Did the reduction in context length result in more coherent questions responses from the Accuser and Intel agents, or did the loss of "raw" dialogue history make their behavior less natural?
- Based on your observations, do you believe this summarization approach would scale to a game with a much larger suspect pool (e.g., `suspect-count=20`)? Explain your reasoning. Feel free to reinforce your hypothesis with additional experimental evidence.

2.3 Implementing Reasoning Agents

To further enhance agent performance, we will implement internal reasoning mechanisms, allowing agents to generate latent "thoughts" before producing their final output. This approach is analogous to the "Chain-of-Thought" techniques explored in previous exercises on prompting for reasoning.

Instructions:

- Implement the `take_turn_thought` method within both the `IntelAgent` and `AccuserAgent` classes in `agents.py`.
- Refine the agent system prompts to explicitly encourage a "Chain-of-Thought" reasoning process before selecting an action.
- Ensure that the **final output remains compatible with the Game Environment's expected schema**, regardless of the internal reasoning steps generated.
- Address the specific failure modes observed in Task 1 by utilizing prompt engineering to guide the agents' logical flow and decision-making.
- To enable this mode, execute the simulation using the `--allow-internal-thought` flag. Note that this feature can be integrated with the `SummarizerAgent` implemented in Task 2.

Analysis:

- Detail your architectural decisions regarding prompt engineering. What specific intermediate reasoning steps did you introduce to guide the agents, and what did you want each agent to analyze internally before generating its environment-compliant response?
- Evaluate whether this "Chain-of-Thought" mechanism successfully mitigated the Task 1 failure modes. Did adding this internal reasoning step improve agent coordination, action-space utilization, or overall success rate compared to the naive baseline?
- If the agents still fail to identify the culprit, discuss the underlying causes of these failures. If they are successful, scale the complexity by increasing the suspect count (and adjusting the turn limit accordingly). Determine the maximum number of suspects your agents are capable of successfully processing, subject to your selected turn limit and applicable API rate restrictions. In scenarios with larger suspect pools, analyze the factors that contribute to model failure.
- Propose three modifications that could further enhance agent performance and improve the overall success rate of the system.

3 Textual Search (30 pt)

We have discussed textual search as an NLP task in various points throughout the class. The goal of textual search is, given a textual query, to search the most relevant text passages in a large corpus. An important example comes from Question Answering - many modern QA systems are a compound of an information retrieval (IR) model and a reading comprehension (RC) model. The purpose of the IR model is, given the question that has been asked, to find the most relevant passages appearing in the web (e.g. Wikipedia), which are likely to contain the answer. Then, the purpose of the RC model is to extract the specific answer out of them.

In this exercise you will explore two widely used NLP methods for textual search: sparse search and dense search. Please upload the `textual_search.ipynb` file into colab. First, run the cells of code which download and construct the corpus. Then, implement the code according to the following instructions.

3.1 Sparse search with TF-IDF

Sparse search is largely based on keyword matching between the query and the documents in the corpus. To quantify the relevance of a document in the corpus to the query, a common measure is TF-IDF, which stands for Term Frequency-Inverse Document Frequency.

In this exercise, you may need to divide sentences into words. You are not allowed to use existing functions in any package for this (such as tokenizers) and must implement a function yourself. However, you may use packages like `re` and `numpy`.

We will use t to refer a single token/word. C will stand as our corpus, which is formally a set of documents. We will use d to denote a single document from the corpus.

1. Term Frequency (TF). Suppose we have a set of English text documents and wish to rank which document is most relevant to the query, "What is the capital of New Zealand?". A simple way

to start out is by eliminating documents that do not contain all three words “capital”, “New”, and “Zealand”, but this might still leave many irrelevant documents (depending on our corpus). To further distinguish between them, we might count the number of times each term occurs in each document; the number of times a term occurs in a document is called its term frequency, and denoted by TF . Formally, if we denote the number of times t appears in d as $count(t, d)$, and the total number of tokens in d by $size(d)$, we get: $TF(t, d) = \frac{count(t, d)}{size(d)}$.

Implement the function `computeTF`, which given a token and a document, computes the corresponding TF measure.

2. Document Frequency (DF). This measures the importance of document in whole set of corpus, this is very similar to TF . The only difference is that TF is frequency counter for a term t in document d , where as DF is the count of occurrences of term t in the document set C . In other words, DF is the number of documents in which the word is present. Formally, $DF(t, N) = \sum_{d \in N} 1_{t \in d}$.¹ We add one occurrence if the term appears in a document at least once, we do not need to know the number of times the term appears.

Implement the function `computeDF` which given a token, and a corpus (which is a list of documents), computes the token's DF .

3. Inverse Document Frequency (IDF). While computing TF , all terms are considered equally important. However it is known that certain terms, such as “is”, “of”, and “that”, may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones which may be more indicative of related documents when they co-occur, by computing the IDF , an inverse document frequency factor is incorporated which diminishes the weight of terms that occur very frequently in the document set and increases the weight of terms that occur rarely. IDF is the inverse of the document frequency which measures the informativeness of term t . When we calculate IDF , it will be very low for the most frequent words such as stop words (because stop words such as “is” is present in almost all of the documents and thus give very little information when they appear, and $\frac{size(C)}{DF}$ will assign a very low value for that word). This finally gives what we want, a relative weighting. Formally, we would like to define it as: $IDF(t) = \frac{size(C)}{DF(t)}$. Now there are few other problems with this, in case of a large corpus, say one with 100,000,000 documents, the IDF value explodes. To avoid the effect we take the log of IDF . During the query time, when a word which is not in vocabulary appears, the DF will be 0. As we cannot divide by 0, we smooth the value by adding 1 to the denominator. Thus, we arrive to the final formula: $IDF(t) = \log \frac{size(C)}{DF(t)+1}$.

Implement the function `computeIDF` which given a token and a corpus, computes the corresponding IDF of the token.

4. We are finally ready to compute $TF - IDF$, our measure that evaluates how important a word is to a document in a corpus. There are many different variations of $TF - IDF$ but for now let us concentrate on the this basic version. $TFIDF(t, d) = TF(t, d) * IDF(t, C)$.

Implement the function `computeTFIDF` which given a token, a document and a corpus, computes the corresponding $TFIDF$ of the token and the document.

5. **Query mechanism:** Write a function that receives a textual query and a textual corpus and returns the top-3 documents with the highest TF-IDF scores, using the function you've implemented above.

¹ 1_b is 1 when the boolean condition b is true, and 0 otherwise.

3.2 Dense search with encoder models

Dense search takes a different approach to textual search. It embeds all of the documents in the corpus into vectors in $\mathbb{R}^d$ (thus creating a dataset of vectors), then embeds the query in $\mathbb{R}^d$ in the same way, and then measures the relevance of a document to the query by a geometric similarity measure between their vector embeddings. Some commonly used geometric similarity measures are the Euclidean distance, the cosine similarity, and the inner product. In this exercise we will use the inner product.

We will embed the texts in $\mathbb{R}^d$ using a pre-trained transformer-based encoder-only model similar to BERT which we have seen in class. The specific model we will use is `all-mpnet-base-v2` which is available in the `sentence_transformers` library. The model is built on the [MPNet](#) architecture and is finetuned using the [Sentence-BERT](#) method. The embedding dimension is $d = 768$.

1. **Ingestion:** First, we will “ingest” the corpus into a vector dataset. This part runs only once prior to receiving any queries. Then the pre-ingested dataset can be used for all queries.

Write the code that loads the `all-mpnet-base-v2` embedding model and embeds the textual corpus we have created above into a numpy matrix called `embedded_corpus`. The matrix should have a row per document in the corpus and $d = 768$ columns. Each row should be set to the vector embedding of the corresponding document in the corpus.

2. **Query mechanism:** Write a function that received a textual query and the embedded corpus, embeds the query into a vector with the same embedding model, and returns the top-3 documents in the corpus as measured by the inner product between their vectors and the query vector.

One advantage of dense search with vectors is the parallelism possible with vector operations. Make sure that your query mechanism is vectorized. Iterative solutions (that compute the inner products one by one) will receive only partial credit.

3.3 What you need to submit

Include the code you’ve written in your submission. In addition, in the included PDF, include the answers to the following questions:

1. Consider the following three queries:

- (a) *What is the capital of New Zealand?*
- (b) *What currency is used in New Zealand?*
- (c) *Should I visit New Zealand if I am interested in sightseeing?*

For each query, list the top-3 results with sparse search and the top-3 results with dense search, along with their scores (TF-IDF scores for sparse search, and inner product scores for dense search).

2. Sparse and dense search both have pros and cons. In what situations would you expect sparse search to be better and in what situations would you expect dense search to be better?
 - In this question, refer only to the quality and relevance of the search results, and disregard computational aspects (i.e., which method is more efficient).
 - **Answer is no more than 4 sentences.**

4 AI Disclosure & Reflection

This question is mandatory, but it will not be graded. Furthermore, its content does not impact the grading and evaluation of the rest of your submission. The goal is to promote transparency and responsible LLM and AI usage.

Answer the following parts briefly (1 paragraph):

1. Did you use AI in the work on this assignment? If yes, please name the specific tools/models used (e.g., ChatGPT—model name, Copilot, Claude, etc.).
2. How and why did you use it – which parts of the work and for what purpose (e.g., brainstorming, outlining, code generation, debugging, editing, evaluation).
3. Reflection: What helped or hindered? What is your takeaway from this experience?

If you did not use AI, write: “No AI used”, and add a sentence explaining why.

References

- Barbi, Ohav, Ori Yoran, and Mor Geva (2025). “Preventing Rogue Agents Improves Multi-Agent Collaboration”. In: *arXiv preprint arXiv:2502.05986*. Accepted to REALM at ACL 2025. URL: <https://arxiv.org/abs/2502.05986>.
- Geva, Mor et al. (2021). “Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies”. In: *Transactions of the Association for Computational Linguistics* 9, pp. 346–361. DOI: [10.1162/tacl.a-00370](https://arxiv.org/abs/2101.02235). URL: <https://arxiv.org/abs/2101.02235>.